Midterm: CI/CD Pipeline with Docker, Kubernetes, Jenkins, and SonarQube

1,000 Points Possible

3/29/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

Details

Objective:

you will set up a **CI/CD pipeline** for a Java application using:

- **Jenkins for automation**
- **SonarQube for static code analysis**
- **Docker for containerization**
- **Kubernetes for orchestration and deployment**

Understanding the CI/CD Pipeline Architecture

What are we building?

You will create a **Jenkins pipeline** that does the following:

1. **Checkout Code** → Pull the latest Java code from GitHub.
2. **Build the Java Application** → Compile using **Java 17**.
3. **Run Unit Tests** → Test using **Java 11**.
4. **Analyze Code Quality** → Use **SonarQube and Java 8**.
5. **Build Docker Image** → Package the Java application inside a Docker container.
6. **Push to Docker Registry** → Store the image in **Docker Hub**.
7. **Deploy to Kubernetes** → Deploy the containerized application using **Kubernetes manifests**.

Part 2: Setting Up the Infrastructure

Create a Custom Docker Network

Ensure all services communicate via the same network.

```
docker network create ci_network
```

Run Jenkins in a Docker Container

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 --network ci_network \
    -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

- **Unlock Jenkins**:

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

- **Install Required Plugins or software**:
  - Pipeline
  - SonarQube Scanner
  - Docker Pipeline
  - Kubernetes Continuous Deploy

Run SonarQube in a Docker Container

```
docker run -d --name sonar -p 9000:9000 --network ci_network sonarqube:lts
```

- **Access SonarQube**: `http://localhost:9000`
- **Default Login**: `admin/admin`

Create Three Docker Containers with Different Java Versions

These simulate different environments for build, testing, and analysis.

```
docker run -d --name java8 --network ci_network openjdk:8-jdk
docker run -d --name java11 --network ci_network openjdk:11-jdk
docker run -d --name java17 --network ci_network openjdk:17-jdk
```

Part 3: Setting Up Kubernetes

Install Kubernetes (Minikube or k3s)

Ensure **Kubernetes** is running on your system:

```
minikube start
kubectl get nodes
```

Create a Kubernetes Deployment for the Java App

Create a file `deployment.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: java-app
  template:
    metadata:
      labels:
        app: java-app
    spec:
      containers:
        - name: java-app
          image: YOUR_DOCKER_HUB_USERNAME/java-app:latest
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: java-app-service
spec:
  selector:
    app: java-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: LoadBalancer
```

Apply the deployment:

```
kubectl apply -f deployment.yaml
```

Part 4: Creating the Jenkins Pipeline

What is a Jenkinsfile?

A **Jenkinsfile** automates the **CI/CD pipeline**.

Jenkinsfile

Create a file named `Jenkinsfile` in the project directory.

```
pipeline {
  agent any

  environment {
    SONARQUBE_SERVER = 'http://sonar:9000'
    DOCKER_IMAGE = 'YOUR_DOCKER_HUB_USERNAME/java-app:latest'
  }

  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/spring-projects/spring-petclinic.git'
      }
    }

    stage('Build') {
      steps {
        script {
          docker.image('openjdk:17-jdk').inside {
            sh 'mvn clean package'
          }
        }
      }
    }

    stage('Test') {
      steps {
        script {
          docker.image('openjdk:11-jdk').inside {
            sh 'mvn test'
          }
        }
      }
    }
```

```
        stage('Static Code Analysis') {
            steps {
                script {
                    docker.image('openjdk:8-jdk').inside {
                        sh 'mvn sonar:sonar -Dsonar.host.url=${SONARQUBE_SERVER}'
                    }
                }
            }
        }

        stage('Build Docker Image') {
            steps {
                script {
                    sh 'docker build -t ${DOCKER_IMAGE} .'
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                script {
                    sh 'docker login -u YOUR_DOCKER_HUB_USERNAME -p YOUR_PASSWORD'
                    sh 'docker push ${DOCKER_IMAGE}'
                }
            }
        }

        stage('Deploy to Kubernetes') {
            steps {
                script {
                    sh 'kubectl apply -f deployment.yaml'
                }
            }
        }
    }
}
```

Part 5: Running the Pipeline in Jenkins

Create a New Jenkins Job

1. Open **Jenkins** (`http://localhost:8080`).
2. Click **"New Item"** → Select **"Pipeline"**.
3. Under **Pipeline Definition**, choose **Pipeline script from SCM**.
4. Enter **GitHub repository URL**.
5. Click **Save** and then **Build Now**.

6. Run SonarQube Container

Pull image:

```
docker pull sonarqube
```

Run container:

```
docker run -d \
--name sonarqube \
-p 9000:9000 \
--network cicd-network \
sonarqube
```

Access dashboard:

http://localhost:9000

Login:

```
username: admin
password: admin
```

Create a **SonarQube token**:

My Account → Security → Generate Token

7. Create Java Environment Containers

You will simulate **three environments**.

Java 17 Build Container

```
docker run -dit \
--name java17-builder \
--network cicd-network \
openjdk:17
```

Java 11 Test Container

```
docker run -dit \
--name java11-tester \
--network cicd-network \
openjdk:11
```

Java 8 Analysis Container

```
docker run -dit \
--name java8-analyzer \
```

```
--network cicd-network \
openjdk:8
```

8. Install Kubernetes Cluster

Install **Minikube**.

Start cluster:

```
minikube start
```

Verify:

```
kubectl get nodes
```

Expected:

```
NAME STATUS
minikube Ready
```

---

9. Create Kubernetes Deployment

Create:

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-app
spec:
  replicas: 2

  selector:
    matchLabels:
      app: java-app

  template:
    metadata:
      labels:
        app: java-app

    spec:
      containers:
      - name: java-app
        image: yourdockerhub/java-app:latest

        ports:
        - containerPort: 8080
```

Create Service

Add:

YAML

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: java-service

spec:
  type: NodePort

  selector:
    app: java-app

  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30007
```

Apply:

kubectl apply -f deployment.yaml

Check:

kubectl get pods
Final Architecture

```
            GitHub
              |
              ▼
            Jenkins
        ┌─────┼─────┐
        ▼     ▼     ▼
    Java17  Java11  Java8
     Build   Test  SonarQube
        |     |     |
        └─────┼─────┘
              |
              ▼
            Docker
              |
              ▼
          Docker Hub
              |
              ▼
          Kubernetes
           (Minikube)
              |
              ▼
          Running Pods
```

Real Industry Improvements

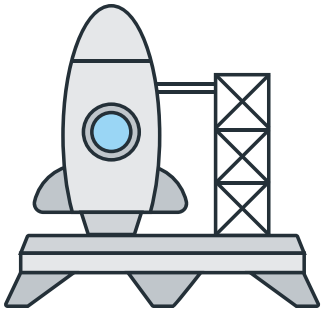
Add:

- Helm charts for Kubernetes
- Jenkins agents instead of containers
- GitOps using Argo CD
- security scanning using Trivy
- monitoring via Prometheus and Grafana

Choose a submission type

Upload

More



Choose a file to upload

or

Webcam Photo

Canvas Files

(https://canvas.csudh.edu/courses/95556/modules/items/3869105)

Submit Assignment